ChatGPT said:

Asymptotic Notation — From the Very Beginning

Let's start from zero. You don't need any prior knowledge.

1. First, why do we need it?

Imagine you write a C++ program to search for a number in an array.

You have two algorithms:

- **Algorithm A:** Finds the number by checking elements one by one.
- **Algorithm B:** Finds the number by repeatedly dividing the search area in half (binary search, on a sorted array).

Both algorithms solve the same problem, but they work differently.

Now imagine searching through:

- 10 numbers
- 1,000 numbers
- 1,000,000 numbers

Which algorithm will perform better when the input becomes very large?

This is why we study algorithm complexity.

We want to understand how an algorithm's work grows when the input size increases.

We call the input size **n**.

For example:

- If an array contains 10 elements, `n = 10`.
- If an array contains 1,000 elements, `n = 1000`.

2. What is asymptotic notation?

Suppose you write a program that processes `n` elements.

You want to describe how its work grows as `n` becomes very large.

Asymptotic notation gives us a mathematical way to describe that growth.

There are three important notations:

1. **Big O — O**
2. **Big Omega — Ω**

3. **Big Theta — Θ**

Let's understand why we need each one.

---

3. Big O — O

Big O describes an upper bound on how an algorithm's work grows.

Imagine an algorithm that checks every element in an array.

```
for (int i = 0; i < n; i++)
{
    // Check element
}
```

If there are 10 elements, the loop runs 10 times.

If there are 100 elements, the loop runs 100 times.

If there are 1,000 elements, the loop runs 1,000 times.

The work grows with the input size.

We describe this growth as **O(n)**.

Why do we need Big O?

We use it to describe an upper limit on the growth of an algorithm's work.

For example, if the algorithm's worst-case work grows linearly, we commonly describe its worst-case complexity as O(n).

Remember: Big O = upper bound.

---

4. Big Omega — Ω

Now imagine the same array-searching problem.

You want to find the number `10`.

Your array is:

```
[10, 20, 30, 40, 50]
```

Your algorithm checks the first element.

It finds `10` immediately!

It doesn't need to check the remaining elements.

This takes constant work, regardless of how large the array is, assuming the target is at the first position.

We describe this best-case growth as **Ω(1)**.

Why do we need Big Omega?

It helps us describe a lower bound on how an algorithm's work grows.

For example, an algorithm might need only constant work in its best case, even when the input is very large.

Remember: Big Omega = lower bound.

Important: Ω does not automatically mean best case. It describes a lower bound; we often use it to discuss best-case growth.

---

5. Big Theta — Θ

Consider this code:

```
for (int i = 0; i < n; i++)
{
    // Process element
}
```

Suppose the loop processes every element exactly once.

Its work grows proportionally to `n`.

It has both:

- An upper bound proportional to `n`.
- A lower bound proportional to `n`.

Therefore, its tight asymptotic bound is **Θ(n)**.

Why do we need Big Theta?

Sometimes, we want to describe the growth rate precisely, within constant factors.

Big Theta tells us that the growth is bounded both above and below by the same rate.

Remember: Big Theta = tight bound.

---

6. Understand all three with one example

Imagine a linear search algorithm that looks for a number in an array of 1,000 elements.

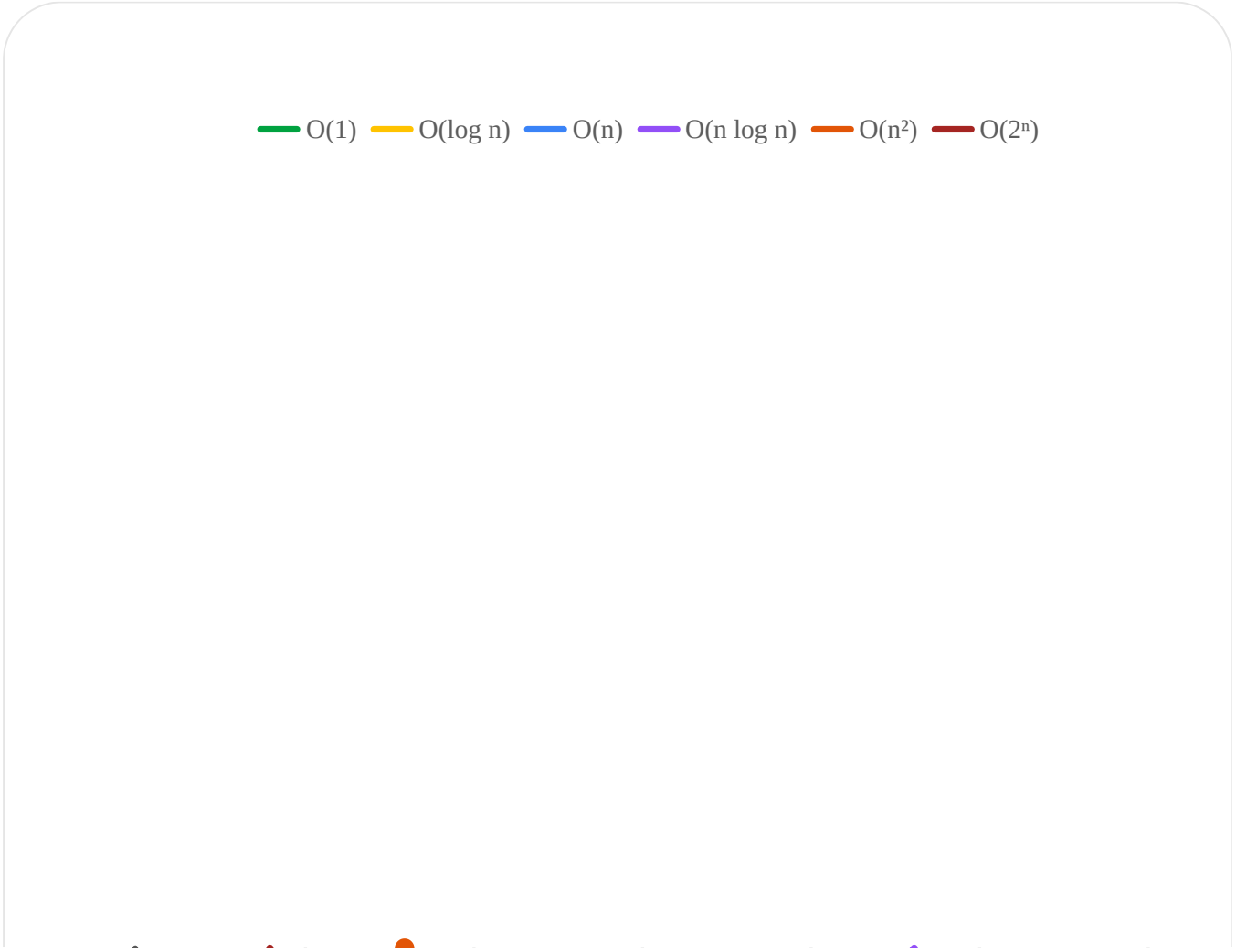
Situation	Elements checked	Complexity
Target is the first element	1	Best case: Θ(1)
Target is the last element	1,000	Worst case: Θ(n)
Target is absent	1,000	Θ(n)

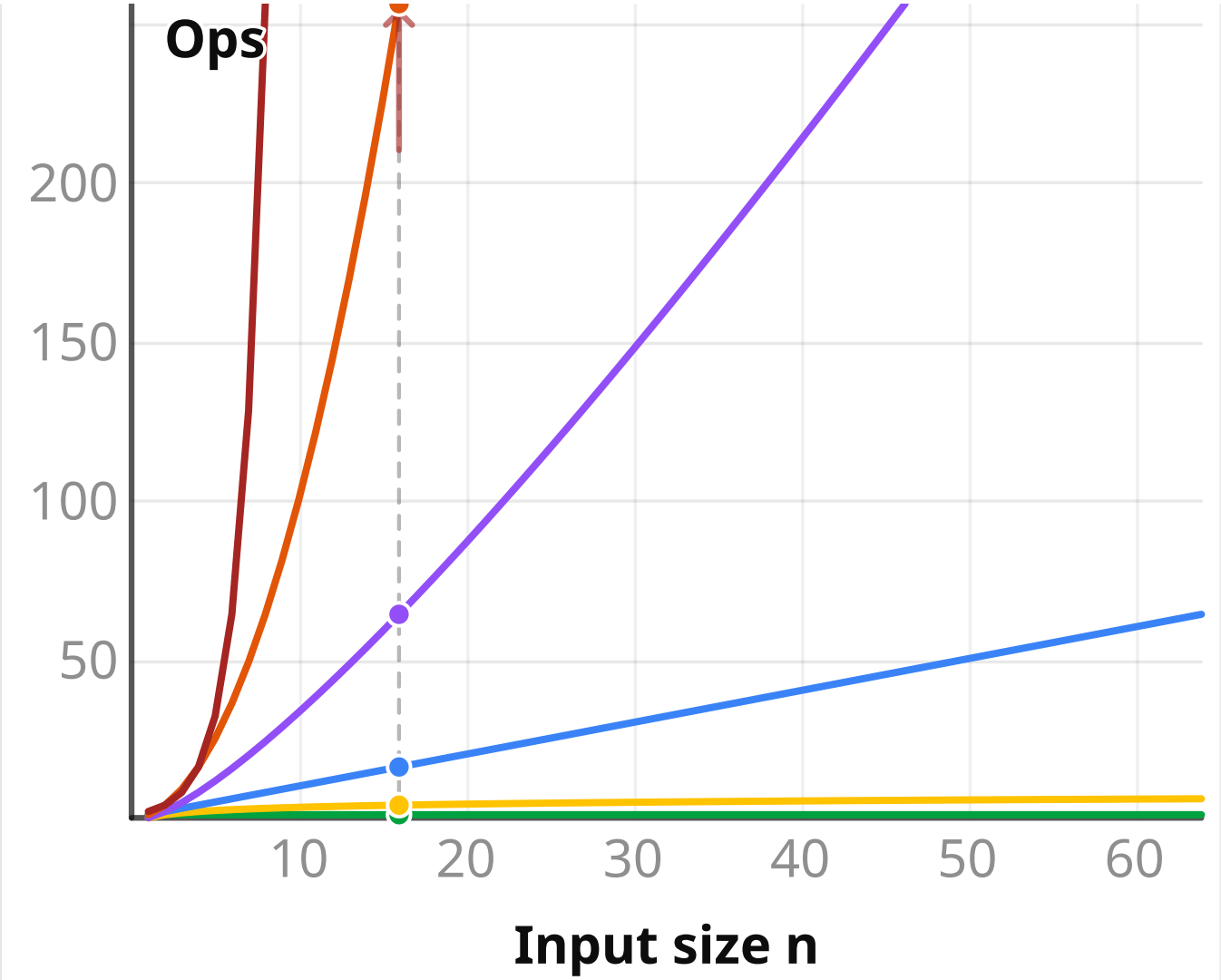
Here is the important distinction:

- **O(n):** An upper bound on the growth of the work.
- **Ω(1):** A lower bound can describe the constant-work best case.
- **Θ(n):** A tight bound describes linear growth when every element must be checked.

Notice that these notations are not three different algorithms. They describe mathematical bounds on algorithmic growth.

7. The easiest way to remember





$$n = 16: \quad (1, \log_2 n, n) = (1, 4, 16)$$
$$(n \log_2 n, n^2) = (64, 256)$$
$$2^n = 65,536$$

n 4 8 16 32 64

Give feedback

Notation	Think of it as	Main question
Big O — O	Upper bound	How fast can the work grow, at most?
Big Omega — Ω	Lower bound	What lower bound can we establish on the work?
Big Theta — Θ	Tight bound	What growth rate bounds the work both above and below?

Final takeaway

We need asymptotic notation because we want to compare algorithms based on **how their work grows as the input becomes larger**, rather than relying only on execution time on one computer.

- **O** describes an upper bound.
- **Ω** describes a lower bound.
- **Θ** describes a tight bound.

That's the foundation. Once you understand these three ideas, learning algorithm time complexity becomes much easier.

If you want, I can:

- ↳ Explain Big O, Big Omega, and Big Theta with a new example algorithm
- ↳ Detail how to analyze algorithm complexity step-by-step
- ↳ Compare linear search and binary search complexities with examples